ECM2433 Sample Paper

The C Family

On Campus Exam

Closed Book

Additional Materials: None

Permitted Materials: No Calculators Permitted

Guidelines:

The duration of this paper is 2hr 00.

Word Count: No Word Count.

Write all answers in the answer booklet provided.

Do not open the exam paper until instructed by the Invigilator.

Use only black pen or pencil.

Mobile and electronic devices are not allowed.

Do not communicate with other candidates during the exam.

Exam Instructions: Answer ALL questions.

This version has the questions first and the skeleton answers at the end.

Question 1

(a) A programmer uses the following macro construction to find the square of two numbers. Using this macro, what is the result of running the code below? Explain how the macro might be modified to yield the square of k + 1. Give an advantage and a disadvantage of this method compared with defining an equivalent function. (8 marks)

```
#define square(x) x * x

int k = 8;
printf("%d\n", square(k + 1));
```

(b) Give the definition of a C preprocessor macro swap(a, b) which interchanges its int arguments. (4 marks)

(c) Given the declarations below, write code to allocate memory and then copy str into dup. You may use malloc, but no other variables or functions. (8 marks)

```
char str[] = "By Divine Aid";
char *dup, *p, *q;
```

(d) Write a function int **alloc2d(int rows, int cols) to allocate a two-dimensional array of integers. (6 marks)

(e) Explain the difference between global and static variables in C. (5 marks)

(f) A programmer writes the following code but is surprised to see no output until the program exits. Explain why and how the result of each printf can be seen immediately. (6 marks)

```
int i;
for (i = 0; i < 10; i++)
    printf("%d ", i);
sleep(10);
exit(0);
```

(Total 37 marks)

Question 2

A class is defined as follows:

```
class ClockType
{
    unsigned int hours;
    unsigned int minutes;
    unsigned int seconds;

protected:
    char *maker;
```

```
public:
    unsigned int seconds_since_midnight()
    {
        return seconds + minutes * 60 + hours * 60 * 60;
    }
};
```

(a) Give code to derive a class, Watch, from the ClockType base class. Your Watch class should have an additional member function printMaker that prints the maker string. (4 marks)

(b) Explain why the Watch class can access the maker attribute, but not the hours, minutes and seconds variables. (4 marks)

(c) Why might the designer of the ClockType class prefer to make the hours, minutes and seconds variables inaccessible to derived classes? (4 marks)

(d) The programmer of another class wishes to obtain and change the hours, minutes and seconds variables. Suggest two mechanisms that could be used to accomplish this. (4 marks)

(e) A further class, SmartWatch, is derived from the Watch class with access modifier public. State the access control of each of the variables and member functions of the ClockType class in the SmartWatch class. (4 marks)

(Total 20 marks)

Question 3

(a) Consider the Rust code below. Explain why it fails to compile, then explain how Rust's approach to memory management here differs from copying a pointer in C. (4 marks)

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{}", s1);
}
```

(b) Consider the Rust code below. State which assignment performs a copy and which performs a move, explain why the program does not compile, and suggest a modification that preserves printing all four values. (6 marks)

```
fn main() {
    let x: i32 = 10;
    let y = x;

    let s1 = String::from("hi");
    let s2 = s1;

    println!("{x} {y} {s1} {s2}");
}
```

(c) In C, iterating over an array typically requires manual index management, while Rust can iterate over references. Give two potential errors with C's manual indexing that Rust's iterator approach prevents, state the type of x in the Rust loop, and write one iterator expression that takes a Vec<i32> called numbers and produces a new Vec<i32> containing only the even numbers, each multiplied by 2. (9 marks)

```
int arr[] = {1, 2, 3, 4, 5};
for (int i = 0; i < 5; i++) {
    printf("%d\n", arr[i]);
}
```

```
let arr = [1, 2, 3, 4, 5];
for x in &arr {
    println!("{}", x);
}
```

(Total 19 marks)

Question 4

(a) Memory management is handled differently in C, C++, and Rust. In C, give two errors that can occur if free is used incorrectly and explain the consequences. Explain RAII in C++ and how it helps prevent those errors. Explain how Rust's ownership system achieves memory safety without a garbage collector, referring to the three ownership rules. Finally, identify the Rust analogues of std::unique_ptr and std::shared_ptr and state a key difference in how Rust enforces correct usage. (15 marks)

(b) Error handling is approached differently in C, C++, and Rust. Give two disadvantages of C's return-code approach, give one advantage and one disadvantage of C++ exceptions, and explain how Rust's Result<T, E> and ? operator combine explicit error types with concise propagation. (9 marks)

```
FILE *f = fopen("data.txt", "r");
if (f == NULL) {
    /* handle error */
}
```

```
use std::fs::File;
use std::io::{self, Read};

fn read_file(path: &str) -> Result<String, io::Error> {
```

```
    let mut file = File::open(path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    Ok(contents)
}
```

(Total 24 marks)

Answers and marking notes

Equivalent correct code and reasoning should receive credit.

Question 1

(a) Question (8 marks): A programmer uses the following macro construction to find the square of two numbers. Using this macro, what is the result of running the code below? Explain how the macro might be modified to yield the square of k + 1. Give an advantage and a disadvantage of this method compared with defining an equivalent function.

Code from question:

```
#define square(x) x * x

int k = 8;
printf("%d\n", square(k + 1));
```

Answer: The macro expands textually, so square(k + 1) becomes k + 1 * k + 1. With k = 8 this evaluates as 8 + 1 * 8 + 1 = 17, not 81.

Answer: #define square(x) ((x) * (x))

Answer: The parentheses force each argument and the whole expression to be grouped correctly. An advantage of a macro is that there is no function call overhead. A disadvantage is that macro arguments can have side effects because they may be evaluated more than once, for example square(i++).

(b) Question (4 marks): Give the definition of a C preprocessor macro swap(a, b) which interchanges its int arguments.

```
#define swap(a, b) do { \
    int tmp = (a);     \
    (a) = (b);         \
    (b) = tmp;         \
} while (0)
```

Answer: A simpler block macro using a temporary int also receives credit. The temporary stores one value while the two arguments are exchanged.

(c) Question (8 marks): Given the declarations below, write code to allocate memory and then copy str into dup. You may use malloc, but no other variables or functions.

Code from question:

```
char str[] = "By Divine Aid";
char *dup, *p, *q;
```

```
#include <assert.h>
#include <stdlib.h>

for (p = str; *p; p++)
    ;

dup = malloc((size_t)(p - str + 1) * sizeof *dup);
assert(dup != NULL);

for (p = str, q = dup; *p; )
    *q++ = *p++;

*q = '\0';
```

Answer: Marks are for finding the string length with pointer arithmetic, allocating space for the terminator, checking malloc did not return NULL, copying with pointer increments, and writing the final null terminator.

(d) Question (6 marks): Write a function int **alloc2d(int rows, int cols) to allocate a two-dimensional array of integers.

```
#include <assert.h>
#include <stdlib.h>

int **alloc2d(int rows, int cols)
{
    int **a = malloc((size_t)rows * sizeof *a);
    assert(a != NULL);

    a[0] = malloc((size_t)rows * (size_t)cols * sizeof **a);
    assert(a[0] != NULL);

    for (int r = 1; r < rows; r++)
        a[r] = a[0] + r * cols;

    return a;
}
```

Answer: Answers that allocate each row separately are also acceptable. Award credit for allocating the row-pointer array, allocating rows * cols integers or separate rows, setting each row pointer correctly, and checking allocation failure.

(e) Question (5 marks): Explain the difference between global and static variables in C.

Answer: A global variable has global scope and can be accessed from other code where it is visible. A static local variable has scope limited to the function in which it is declared, but it persists between calls to that function.

(f) Question (6 marks): A programmer writes the following code but is surprised to see no output until the program exits. Explain why and how the result of each printf can be seen immediately.

Code from question:

```
int i;
for (i = 0; i < 10; i++)
    printf("%d ", i);
sleep(10);
exit(0);
```

Answer: printf writes to a buffered stream. The output may stay in stdout's buffer until the buffer is full, a newline is written on a line-buffered terminal stream, or the program exits normally. To force output immediately after each printf, call fflush(stdout).

```
printf("%d ", i);
fflush(stdout);
```

Question 2

Shared code from question:

```
class ClockType
{
    unsigned int hours;
    unsigned int minutes;
    unsigned int seconds;

protected:
    char *maker;

public:
    unsigned int seconds_since_midnight()
    {
        return seconds + minutes * 60 + hours * 60 * 60;
    }
};
```

(a) Question (4 marks): Give code to derive a class, Watch, from the ClockType base class. Your Watch class should have an additional member function printMaker that prints the maker string.

```
class Watch : public ClockType
{
public:
    void printMaker()
    {
        std::cout << maker << std::endl;
    }
};
```

Answer: Marks are for public inheritance and a printMaker function that accesses maker.

(b) Question (4 marks): Explain why the Watch class can access the maker attribute, but not the hours, minutes and seconds variables.

Answer: maker is protected, so it is accessible inside derived classes such as Watch. hours, minutes, and seconds are private because they appear before any public or protected access specifier, so derived classes cannot access them directly.

(c) Question (4 marks): Why might the designer of the ClockType class prefer to make the hours, minutes and seconds variables inaccessible to derived classes?

Answer: Keeping them private hides the implementation and forces users or derived classes to work through the public/protected interface. The implementation can then be changed later without breaking code that uses the class.

(d) Question (4 marks): The programmer of another class wishes to obtain and change the hours, minutes and seconds variables. Suggest two mechanisms that could be used to accomplish this.

Answer: Two possible mechanisms are: provide getter and setter member functions in ClockType, or declare selected functions/classes as friends. Making everything public receives limited credit because it weakens encapsulation.

(e) Question (4 marks): A further class, SmartWatch, is derived from the Watch class with access modifier public. State the access control of each of the variables and member functions of the ClockType class in the SmartWatch class.

Answer: hours, minutes, and seconds remain inaccessible because they are private in ClockType. maker is protected and remains accessible inside Watch and SmartWatch. seconds_since_midnight is public in ClockType, Watch, and SmartWatch.

Question 3

(a) Question (4 marks): Consider the Rust code below. Explain why it fails to compile, then explain how Rust's approach to memory management here differs from copying a pointer in C.

Code from question:

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{}", s1);
}
```

Answer: The code fails because ownership of the String is moved from s1 to s2 by let s2 = s1. After the move, s1 is no longer valid, so println cannot use it.

Answer: In C, copying a pointer would usually create a shallow copy where both pointers refer to the same memory. Rust enforces that a value has only one owner at a time, preventing double frees and dangling pointers.

(b) Question (6 marks): Consider the Rust code below. State which assignment performs a copy and which performs a move, explain why the program does not compile, and suggest a modification that preserves printing all four values.

Code from question:

```
fn main() {
    let x: i32 = 10;
    let y = x;

    let s1 = String::from("hi");
    let s2 = s1;

    println!("{x} {y} {s1} {s2}");
}
```

Answer: let y = x performs a copy because i32 implements Copy. let s2 = s1 performs a move because String owns heap data and does not implement Copy.

Answer: The program does not compile because s1 is used after it has been moved into s2. One fix is to clone s1 before assigning to s2.

Answer: let s2 = s1.clone();

(c) Question (9 marks): In C, iterating over an array typically requires manual index management, while Rust can iterate over references. Give two potential errors with C's manual indexing that Rust's iterator approach prevents, state the type of x in the Rust loop, and write one iterator expression that takes a Vec<i32> called numbers and produces a new Vec<i32> containing only the even numbers, each multiplied by 2.

Code from question:

```
int arr[] = {1, 2, 3, 4, 5};
for (int i = 0; i < 5; i++) {
    printf("%d\n", arr[i]);
}
```

Code from question:

```
let arr = [1, 2, 3, 4, 5];
for x in &arr {
    println!("{}", x);
}
```

Answer: C manual indexing can cause off-by-one errors, such as using <= instead of <, and out-of-bounds access if the hard-coded length does not match the actual array size.

Answer: In the Rust loop, x has type &i32.

```
let result: Vec<i32> = numbers
    .into_iter()
    .filter(|x| x % 2 == 0)
    .map(|x| x * 2)
    .collect();
```

Answer: An answer using .iter() with appropriate dereferencing is also acceptable.

Question 4

(a) Question (15 marks): Memory management is handled differently in C, C++, and Rust. In C, give two errors that can occur if free is used incorrectly and explain the consequences. Explain RAII in C++ and how it helps prevent those errors. Explain how Rust's ownership system achieves memory safety without a garbage collector, referring to the three ownership rules. Finally, identify the Rust analogues of std::unique_ptr and std::shared_ptr and state a key difference in how Rust enforces correct usage.

Answer: Two C errors are double free and use-after-free. A double free can corrupt the heap allocator's internal data structures, causing crashes or security vulnerabilities. A use-after-free happens when memory is accessed after it has been released, giving undefined behaviour.

Answer: RAII means Resource Acquisition Is Initialization. A resource is acquired by an object and released in that object's destructor. When the object goes out of scope, the destructor runs automatically, helping ensure the resource is released exactly once.

Answer: Rust's ownership rules are: each value has an owner; there can only be one owner at a time; when the owner goes out of scope, the value is dropped. The borrow checker enforces these rules at compile time, preventing double-free and use-after-free errors.

Answer: Box<T> is analogous to std::unique_ptr for unique ownership. Rc<T> is analogous to std::shared_ptr for reference-counted shared ownership. Rust enforces ownership and borrowing rules through the type system at compile time, while C++ smart pointers rely more on runtime mechanisms and programmer discipline.

(b) Question (9 marks): Error handling is approached differently in C, C++, and Rust. Give two disadvantages of C's return-code approach, give one advantage and one disadvantage of C++ exceptions, and explain how Rust's Result<T, E> and ? operator combine explicit error types with concise propagation.

Code from question:

```
FILE *f = fopen("data.txt", "r");
if (f == NULL) {
    /* handle error */
}
```

Code from question:

```
use std::fs::File;
use std::io::{self, Read};

fn read_file(path: &str) -> Result<String, io::Error> {
    let mut file = File::open(path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    Ok(contents)
}
```

Answer: Two disadvantages of C return codes are that the programmer can forget to check them, and the same return type often has to represent both successful values and error indicators. Error information can also be limited.

Answer: An advantage of C++ exceptions is that errors cannot be silently ignored if they are not caught; they propagate up the call stack. A disadvantage is that exceptions can make control flow harder to follow and require care with exception safety.

Answer: Rust makes recoverable errors explicit in the function return type with Result<T, E>. The ? operator unwraps Ok values and returns Err values early from the current function. This gives concise propagation like exceptions, but the possible error path remains visible in the type system.